Experiment - 6

Nikhil Kale
D15A/50

Aim - Apply K-Means and Hierarchical Clustering on sample datasets.

1. Dataset Source

Dataset used:
https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python

Title: Mall Customers Dataset

Dataset Description

The dataset contains customer information used for market segmentation analysis.

Number of records: 200 customers
Number of features: 5 attributes

Features:

- CustomerID – Unique identifier (not useful for clustering)
- Gender – Male/Female
- Age – Age of the customer
- Annual Income ($) – Customer's yearly income
- Spending Score (1-100) – Score assigned based on spending behavior

Selected Features for Clustering:

- Annual Income ($)
- Spending Score (1-100)

Dataset Characteristics:

- No missing values
- Mix of categorical and numerical features
- Unsupervised learning problem (no target variable)

Mathematical Formulation

K-Means Clustering

K-Means aims to partition data into K clusters by minimizing intra-cluster variance.

Objective Function:

$$J = \sum_{i=1}^{K} \sum_{x \in C_i} ||x - \mu_i||^2$$

Where:

- Ci = cluster i
- μi = centroid of cluster i

Steps:

1. Initialize K centroids
2. Assign each point to nearest centroid
3. Update centroids
4. Repeat until convergence

Hierarchical Clustering

Builds clusters using a tree structure (dendrogram).

Uses Ward's method:

$$D(A, B) = \frac{|A||B|}{|A| + |B|} ||\mu_A - \mu_B||^2$$

Steps:

1. Start with each data point as a cluster
2. Merge closest clusters
3. Repeat until one cluster remains

Algorithm Limitations

K-Means Limitations

- Requires predefined number of clusters (K)
- Sensitive to initial centroid selection
- Assumes spherical clusters
- Sensitive to outliers

Hierarchical Clustering Limitations

- Computationally expensive for large datasets
- Difficult to decide optimal cluster cut
- Cannot undo merging decisions

Methodology / Workflow

Steps followed in the experiment:

1. Load dataset
2. Perform exploratory data analysis
3. Select relevant features (Income & Spending Score)
4. Apply feature scaling
5. Use Elbow Method to determine optimal K
6. Apply K-Means clustering
7. Visualize clusters
8. Apply Hierarchical clustering using dendrogram
9. Compare both clustering techniques

Workflow:

Dataset → EDA → Feature Selection → Scaling → K-Means → Elbow Method → Hierarchical Clustering → Visualization → Comparison

Performance Analysis

Since clustering is unsupervised, performance is evaluated visually and using clustering structure.

Observations:

- Optimal number of clusters found ≈ 5
- K-Means clearly forms distinct groups based on spending behavior
- Hierarchical clustering confirms similar grouping through dendrogram
- Clusters represent meaningful customer segments

Cluster Interpretation:

1. High Income – High Spending → Target customers
2. High Income – Low Spending → Potential customers
3. Low Income – High Spending → Impulsive buyers
4. Low Income – Low Spending → Low value
5. Average group

K-Means performs faster and produces cleaner cluster boundaries, while hierarchical clustering provides better insight into cluster relationships.

Hyperparameter Tuning

K-Means

Parameter tuned:

- Number of clusters (K)

Method used:

- Elbow Method (WCSS vs K)

Observation:

- WCSS decreases as K increases
- Optimal K chosen where curve bends → K = 5

Hierarchical Clustering

Parameter tuned:

- Linkage method (Ward used)

Observation:

- Dendrogram used to identify optimal number of clusters
- Largest vertical distance indicates cluster separation

Output-

[1]
7s

```python
# Basic Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Clustering
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.preprocessing import StandardScaler

plt.style.use("seaborn-v0_8")
```

[2]
0s

```python
df = pd.read_csv("Mall_Customers.csv")

df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

[3]
0s

```python
df.info()
df.describe()
df.isnull().sum()
```

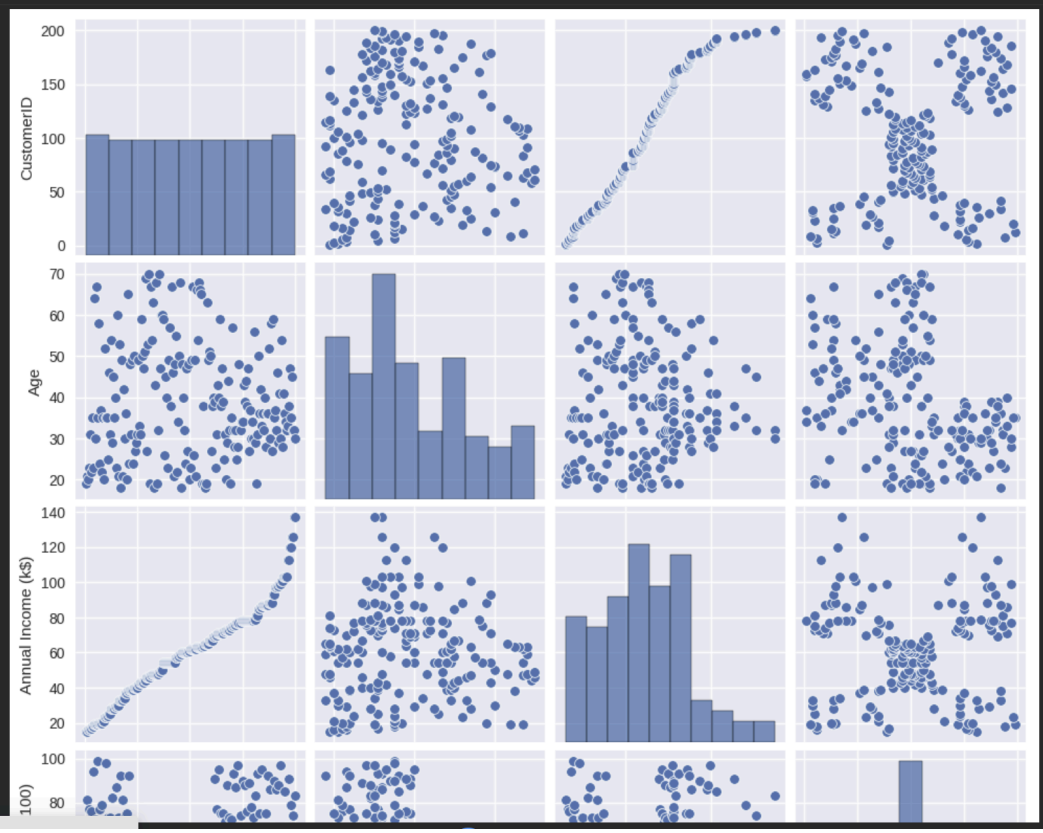
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

	0
CustomerID	0
Gender	0
Age	0
Annual Income (k$)	0
Spending Score (1-100)	0

dtype: int64

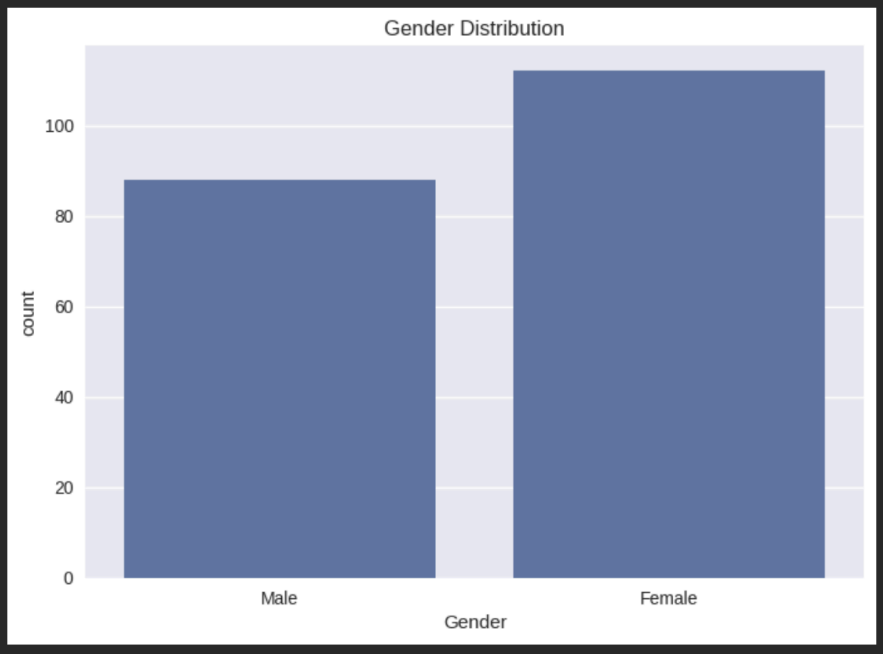
[4]
11s

```python
sns.pairplot(df)
plt.show()
```



[5]
0s

```python
sns.countplot(x='Gender', data=df)
plt.title("Gender Distribution")
plt.show()
```



[6]

```python
plt.figure(figsize=(6,5))
sns.scatterplot(x='Annual Income (k$)', y='Spending Score (1-100)', data=df)
plt.title("Income vs Spending Score")
plt.show()
```

[7]

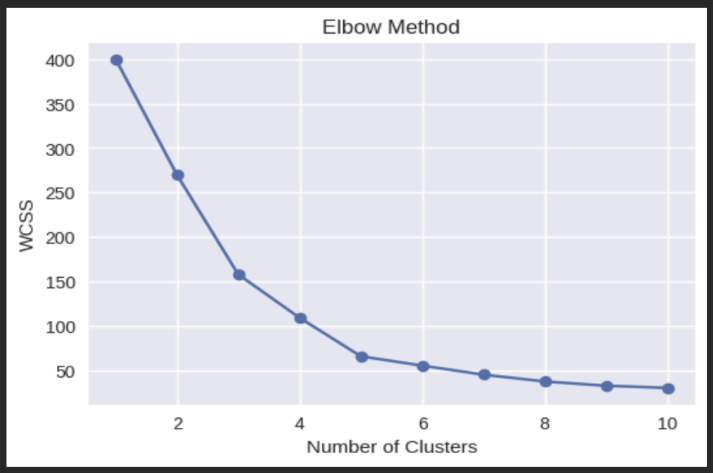
```python
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
```

[8]

```python
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

[9]

```python
wcss = []

for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(6,4))
plt.plot(range(1,11), wcss, marker='o')
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```



[10]

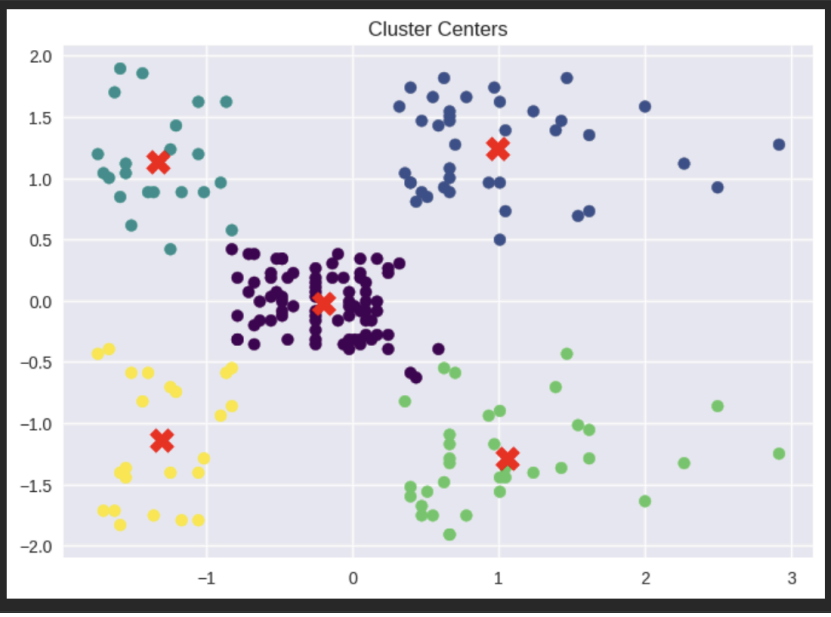
```python
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
y_kmeans = kmeans.fit_predict(X_scaled)
```

[11]
✓ 0s

```python
plt.figure(figsize=(7,5))

plt.scatter(X_scaled[:,0], X_scaled[:,1], c=y_kmeans, cmap='viridis')

plt.title("K-Means Clustering")
plt.xlabel("Annual Income (scaled)")
plt.ylabel("Spending Score (scaled)")

plt.show()
```

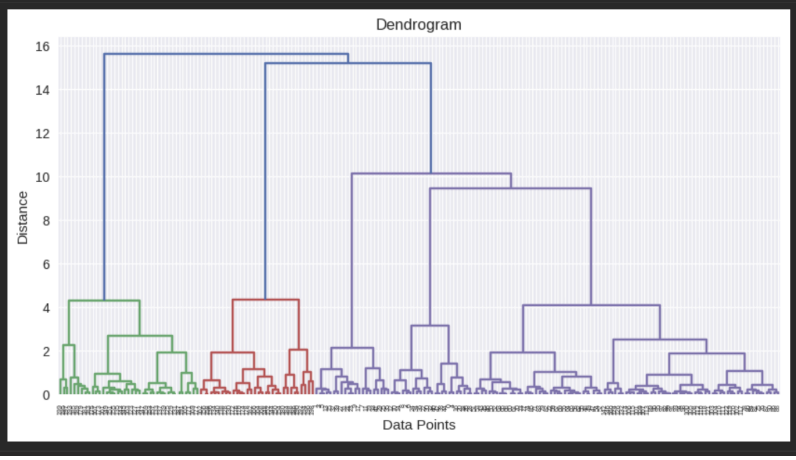
[12]
✓ 0s

```python
centers = kmeans.cluster_centers_

plt.scatter(X_scaled[:,0], X_scaled[:,1], c=y_kmeans, cmap='viridis')
plt.scatter(centers[:,0], centers[:,1], s=200, c='red', marker='X')

plt.title("Cluster Centers")
plt.show()
```



[13]
0s

```python
plt.figure(figsize=(10,5))

linked = linkage(X_scaled, method='ward')

dendrogram(linked)

plt.title("Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()
```

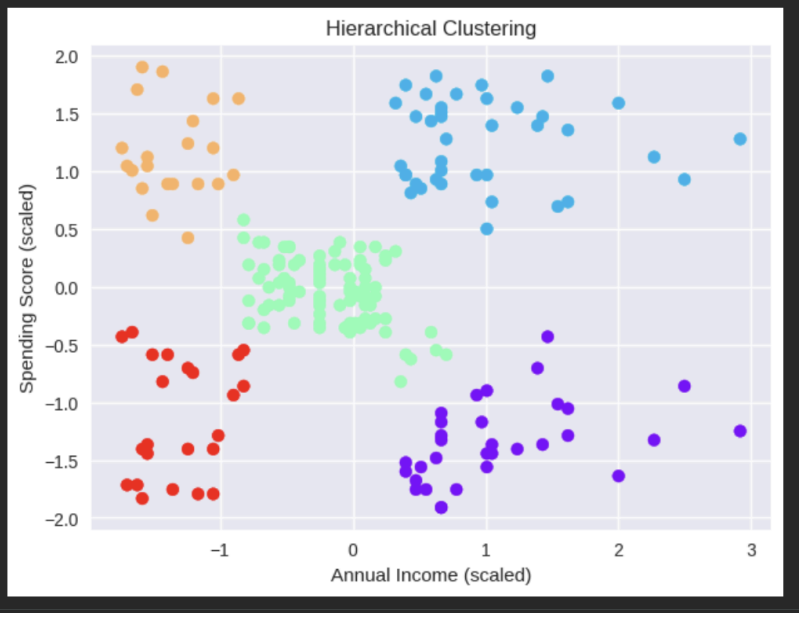


[14]
0s

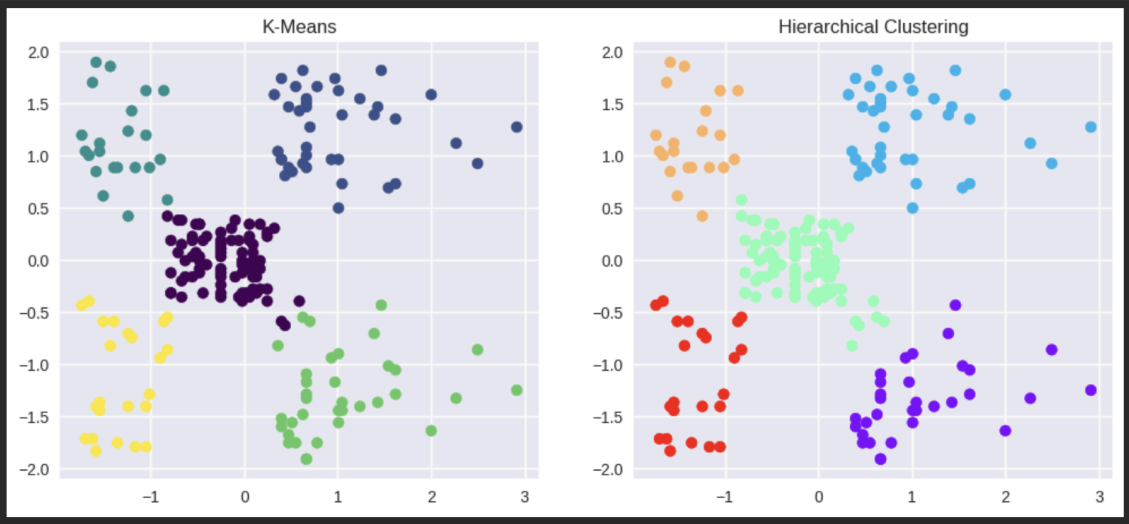
```python
from sklearn.cluster import AgglomerativeClustering

hc = AgglomerativeClustering(n_clusters=5, linkage='ward')
y_hc = hc.fit_predict(X_scaled)
```

[15]
0s

```python
plt.figure(figsize=(7,5))

plt.scatter(X_scaled[:,0], X_scaled[:,1], c=y_hc, cmap='rainbow')

plt.title("Hierarchical Clustering")
plt.xlabel("Annual Income (scaled)")
plt.ylabel("Spending Score (scaled)")

plt.show()
```



```
[16]
✓ 0s
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=y_kmeans, cmap='viridis')
plt.title("K-Means")

plt.subplot(1,2,2)
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=y_hc, cmap='rainbow')
plt.title("Hierarchical Clustering")

plt.show()
```



Conclusion

In this experiment, both K-Means and Hierarchical Clustering were successfully applied to the customer segmentation dataset.

K-Means efficiently grouped customers into 5 distinct clusters based on income and spending behavior, while hierarchical clustering provided a tree-based representation of cluster formation.

The results demonstrate that clustering techniques can effectively identify meaningful customer segments, which can be used for targeted marketing and business decision-making.